SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

MED23CL202 · PYTHON PROGRAMMING LABORATORY

B.Tech Medical Engineering · II Year / III Semester

MINI-PROJECT REPORT

[Title of your project]

e.g. "VitalCheck: a ward vitals monitor that flags abnormal readings"

Track (tick one)

- ☐ **A Patient Vitals Monitor** record vitals, flag abnormal readings
- ☐ **B Pharmacy Stock Manager** dispense / restock, low-stock warnings
- ☐ **C Clinic Appointment Book** book / cancel OPD slots, no double-booking
- ☐ **D Health-Camp Screening Analyzer** classify visitors by BMI and BP
- ☐ **Own topic, approved by faculty:** _____

Submitted by

	Student 1	Student 2 (if working in a pair)
Name		
Register Number		

Programme / Section : .....

Academic Year : .....

Faculty In-charge : .....

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Name(s) : .....

Register Number(s) : .....

Semester / Section : .....

Academic Year : .....

Certified that this is the bonafide record of the mini-project titled "_____" carried out by the above student(s) in the course **MED23CL202 – Python Programming Laboratory** during the academic year _____.

_____ _____

Faculty In-charge **Head of the Department**

Name & Designation Name & Seal

Submitted for the Practical Examination held on _____

_____ _____

Internal Examiner **External Examiner**

DECLARATION

I / We, _____ (Reg. No. _____) and _____ (Reg. No. _____), declare that this mini-project report titled "_____" is our own work, done as part of MED23CL202 – Python Programming Laboratory under the guidance of _____. The program code and results in this report were written, run and tested by us. Any code, data or idea taken from another source is credited in the References.

Place: Chennai
Date: _____ **Signature of the Student(s)**

ACKNOWLEDGEMENT

> **GUIDANCE · delete this box before you submit**
> One short paragraph (5–7 lines). Thank the Dean / Principal, the Head of the Department, your Faculty In-charge, the lab staff, and anyone else who helped. Keep it sincere and specific; do not copy one from the internet.

CONTENTS

1. PROJECT AT A GLANCE 4
Abstract 4
2. THE PROBLEM 4
2.1 Who uses it, and where 4
2.2 What goes wrong today 5
2.3 What my program does 5
3. OBJECTIVES 5
4. DESIGN 5
4.1 The data 5
4.2 Classes 6
4.3 Errors and the custom exception 6
4.4 Data file 7
4.5 Menu and program flow 7
5. HOW THE PROJECT GREW 7
6. COURSE CONCEPTS USED 8
7. KEY CODE EXPLAINED 9
7.1 Class and subclass 9
7.2 Custom exception 9
7.3 Saving and loading the CSV 9
7.4 Analysis and chart 9
8. RESULTS 9
9. TESTING 11
10. LIMITATIONS AND SAFETY 11
11. CONCLUSION AND WHAT I LEARNED 12
11.1 Conclusion 12
11.2 What I learned 12
11.3 Future scope 12
12. REFERENCES 12
APPENDIX A: FULL PROGRAM CODE 14
APPENDIX B: WORK SPLIT (PAIRS ONLY) 15

GUIDANCE · delete this box before you submit

Before you submit, check:

1. Every guidance box (like this one) and every grey "e.g." example is deleted or replaced with your own text.
2. Contents is refreshed: right-click it → Update Field → Update entire table.
3. Your register number is in the footer (double-click the footer to edit it).
4. Line numbers in Appendix A match the ones you quote in Section 6.
5. Every screenshot is readable when printed, and has a caption.

A complete report is usually 15–20 pages including the code. Length earns no marks; clear evidence does. Marks: correctness 40% · code quality & comments 20% · breadth of course concepts 20% · demo & viva 20%.

1. PROJECT AT A GLANCE

> **GUIDANCE · delete this box before you submit**
> Fill this table so the examiner can see your whole project in ten seconds. One line per row.

Item	Your project
Track	*e.g. A – Patient Vitals Monitor*
Who would use it	*e.g. ward nurse on a night shift*
What it does	*e.g. records temperature, pulse and BP; flags abnormal readings; shows each patient's trend*
Class and subclass	*e.g. Patient → ICUPatient (stricter alert limits)*
Custom exception	*e.g. InvalidReadingError*
Data file	*e.g. vitals.csv (name, temp, pulse, bp)*
pandas summary	*e.g. mean / min / max temperature per patient*
Chart	*e.g. line chart of one patient's temperature over time*
Program size	*e.g. 1 file, about 180 lines*

Abstract

> **GUIDANCE · delete this box before you submit**
> 120–150 words, one paragraph. Say what problem your program solves, for whom, the main Python features it uses, and what it can do when finished. Write this LAST.

2. THE PROBLEM

2.1 Who uses it, and where

> **GUIDANCE · delete this box before you submit**
> Describe the real setting in 4–6 lines: the place (ward, pharmacy counter, OPD desk, health camp), the person who would use your program, and what they do today. Example: “At a school health camp, two volunteers screen about 200 children a day and write height and weight in a register by hand…”

2.2 What goes wrong today

> **GUIDANCE · delete this box before you submit**
> 2–4 specific problems your program fixes. Good: “an abnormal BP reading written in the register is noticed only when the doctor reads it at the end of the shift.” Weak: “manual work is slow and error-prone.”

2.3 What my program does

> **GUIDANCE · delete this box before you submit**
> 3–5 sentences in plain words: what the user can do with your program, step by step, and what they get at the end.

3. OBJECTIVES

> **GUIDANCE · delete this box before you submit**
> Rewrite these for your own project. Each objective should be something you can show working in the demo.

The objectives of this mini-project are:

- *To store … records using a class, with validation so impossible values are refused.*
- *To save all records to a CSV file and reload them when the program starts again.*
- *To handle wrong input with try/except and a custom exception, so the program never crashes.*
- *To summarise the data with pandas and show the key pattern in a chart.*

4. DESIGN

4.1 The data

> **GUIDANCE · delete this box before you submit**
> List every field your program stores. For medical values give the unit and the range you accept, and name the source of any normal range (textbook, WHO page, your faculty) in References.

Field	Python type	Unit	Accepted values / rule	Example
temp	*float*	*°F*	*90 to 110; ≥ 100.4 flagged as fever*	*98.6*
pulse	*int*	*beats/min*	*30 to 220*	*72*

4.2 Classes

> **GUIDANCE · delete this box before you submit**
> One row per class. In the last column, say what the subclass does differently (the method it overrides). This is your inheritance and polymorphism.

Class	Inherits from	Attributes	Methods	What is different
Patient	–	*name, __readings (private)*	*add_reading(), status()*	–
ICUPatient	*Patient*	*same*	*status()*	*stricter alert limits*

4.3 Errors and the custom exception

Exception	Raised when	Caught in	Message the user sees
InvalidReadingError	*temperature outside 90–110 °F*	*menu option 1*	*"Temperature must be between 90 and 110 °F"*
ValueError	*user types text for a number*	*read_float()*	*"Please enter a number"*

4.4 Data file

> **GUIDANCE · delete this box before you submit**
> Name of your CSV file, its header row, when it is read (e.g. at start-up) and when it is written (e.g. after every new record, or on exit). Paste the first 3–4 lines of a real file.

4.5 Menu and program flow

Option	What happens	Function / method called
1	*Add a reading for a patient*	*add_reading()*
2	*Show all abnormal readings*	*show_alerts()*
...		
0	*Save and quit*	*save_csv()*

Algorithm

> **GUIDANCE · delete this box before you submit**
> Number the steps of your whole program in plain English, as in your e-record: start with "Step 1: Start" and end with "Step n: Stop". A flowchart is optional; if you draw one, insert it below the algorithm as Figure 4.1.

Step 1: Start

Step 2: …

Step n: Stop

5. HOW THE PROJECT GREW

> **GUIDANCE · delete this box before you submit**
> Your project was built one step after each experiment. In the last column, say in a few words what you actually added to YOUR project that week. If you skipped a step, say so and say where it was added later.

Exp	Concept (CO)	What I added to my project
1	Data types, operators, if/elif/else (CO1)	*e.g. read one patient's vitals and printed NORMAL / FEVER*

Exp	Concept (CO)	What I added to my project
2	Loops, collections, functions (CO1)	
3	Classes, objects, encapsulation (CO2)	
4	Inheritance and polymorphism (CO2)	
5	Reading and writing text files (CO3)	
6	CSV files (CO3)	
7	try / except (CO4)	
8	raise, finally, user-defined exceptions (CO4)	
9	Data analysis with pandas (CO5)	
10	Matplotlib / Seaborn charts (CO5)	

6. COURSE CONCEPTS USED

> **GUIDANCE · delete this box before you submit**
> This table is how the examiner checks "breadth of course concepts" (20% of the marks). For each concept give the function or class where it appears and the line numbers in Appendix A. Leave a row empty if you did not use it; do not claim what is not there.

Concept	Where in my code	Lines (App. A)
Menu loop that runs until the user quits		
Function that returns a value		
List or dictionary		
Class with a private attribute and a validating method		
Subclass that overrides a method		
Records saved to CSV		
Records reloaded from CSV at start-up		
try / except around user input		
Custom exception raised and caught		

Concept	Where in my code	Lines (App. A)
pandas summary (groupby, filter or describe)		
Chart (Matplotlib or Seaborn)		
Anything extra (e.g. list comprehension, finally)		

7. KEY CODE EXPLAINED

> **GUIDANCE · delete this box before you submit**
> Pick the 3–4 most important pieces of your code. Paste each one (5–20 lines), then explain in 2–4 lines what it does and why you wrote it that way. Your full program goes in Appendix A, not here. Expect to be asked about these in the viva.

7.1 Class and subclass

```
# paste your class and subclass here
```

Explain: …

7.2 Custom exception

```
# paste where the exception is defined, raised and caught
```

Explain: …

7.3 Saving and loading the CSV

```
# paste your save and load functions
```

Explain: …

7.4 Analysis and chart

```
# paste your pandas and chart code
```

Explain: …

8. RESULTS

> **GUIDANCE · delete this box before you submit**
> Run your program and paste clear screenshots (crop to the useful part). Under each, write one line saying what it proves. For the chart, also say what it tells the user, e.g. “Bed 4's temperature has risen for three readings in a row.”

Paste screenshot here

Figure 8.1: Main menu and a normal run

Paste screenshot here

Figure 8.2: Wrong input caught (shows your error message, no crash)

Paste screenshot here

Figure 8.3: Data reloaded after restarting the program

Paste screenshot here

Figure 8.4: pandas summary

Paste chart here

Figure 8.5: Chart

9. TESTING

> **GUIDANCE · delete this box before you submit**
> At least 8 tests. Include: normal values, values exactly on a threshold, text where a number is expected, impossible values (e.g. pulse 0, negative quantity), a missing or empty CSV file, and your custom exception. Fill "Actual result" only after running the test.

No.	What is tested	Input / action	Expected result	Actual result	Pass / Fail
1	*normal reading*	*temp 98.6*	*saved, status NORMAL*		
2	*threshold*	*temp 100.4*	*flagged FEVER*		
3	*text for a number*	*temp "abc"*	*"Please enter a number", menu shown again*		
4					
5					
6					
7					
8					

10. LIMITATIONS AND SAFETY

> **GUIDANCE · delete this box before you submit**
> Be honest: what your program cannot do, where its thresholds are simplified, and what could go wrong if someone relied on it. Examiners give credit for knowing the limits of your own work.

This program is a student learning project. It is not a medical device and must not be used to make decisions about real patients.

- *e.g. One fixed fever threshold is used for all ages.*
- *e.g. Data is stored in a plain CSV file with no password, so it is not suitable for real patient records.*

11. CONCLUSION AND WHAT I LEARNED

11.1 Conclusion

> **GUIDANCE · delete this box before you submit**
> One paragraph: which objectives from Section 3 were met, which were only partly met, and why.

11.2 What I learned

> **GUIDANCE · delete this box before you submit**
> Name the hardest bug or problem you met, how you found it, and how you fixed it. Then name one Python concept you understand better because of this project. (Each student in a pair writes their own.)

11.3 Future scope

- *e.g. A simple graphical window (Tkinter) instead of the text menu.*
- *…*

12. REFERENCES

> **GUIDANCE · delete this box before you submit**
> Number every source you used: books, websites, documentation, the source of any normal ranges, and any AI tool that helped you (say what it helped with).

[1] Python Software Foundation, “The Python Tutorial,” docs.python.org, accessed DD Month YYYY.

[2] pandas development team, “pandas documentation,” pandas.pydata.org, accessed DD Month YYYY.

[3] Author(s), Title of Book, Edition. City: Publisher, Year.

APPENDIX A: FULL PROGRAM CODE

> **GUIDANCE · delete this box before you submit**
>
> Paste your complete final program below, in Consolas 9 pt so the indentation stays clear. This page has line numbers switched on; quote them in Section 6.
>
> Comments are part of the marks: one comment above every class and function saying what it does, and short comments on any line that is not obvious.

```
# ================= MINI-PROJECT CODE =================
# paste your program here


```

APPENDIX B: WORK SPLIT (PAIRS ONLY)

> **GUIDANCE · delete this box before you submit**
> Delete this appendix if you worked alone. Otherwise list who did what. Both students must be able to explain the whole program in the viva.

Part of the project	Student 1	Student 2
e.g. Patient class and subclass		